

<BFS>

장점:

- 시작 노드에서 목표 노드까지의 최단 경로를 보장한다.
- 목표 노드가 시작노드와 가까이 있을 경우 빠르게 찾을 수 있다.

단점:

- 큐에 많은 노드를 저장해야 하므로 메모리 사용량이 많다.
- 그래프가 넓을수록 탐색해야 할 노드가 많아져 느려질 수 있다.

특징:

- 큐 자료구조를 사용한다.
- 시작 노드에서 가까운 노드부터 순서대로 탐색한다.
- 같은 레벨의 노드를 모두 방문한 후 다음레벨로 넘어간다.

<DFS>

장점

- 스택 또는 재귀를 사용하므로 메모리 사용량이 적다.
- 경로를 끝까지 탐색함로 미로찾기, 사이클 탐지 등에 유리하다.

단점:

- 최단 경로를 보장하지 않는다.
- 그래프가 깊을 경우 재귀 호출이 많아져 스택 오버플로우가 발생할 수 있다.

특징:

- 스택 또는 재귀를 사용한다.
- 한 방향으로 끝까지 탐색한 후 되돌아와 다른 경로를 탐색한다.
- 모든 경로를 탐색해야 하는 문제에 적합하다.